

SEEKER-120AM GEN-4.5: Academic & Systems Engineering Documentation

Line Spacing: 1.15 | Author: Rüzgar Albayrak | Status: GEN-4.5 Core Engine

ABSTRACT

SEEKER-120AM GEN-4.5 is a deterministic, real-time Command and Control (C2) simulation framework engineered to resolve the computational bottlenecks of stochastic radar tracking. By migrating from an interpreted Python architecture to a pre-allocated, compiled C++20 engine, the system evaluates 12-Degree-of-Freedom (12-DOF) quaternion kinematics and inverse fourth-power radar equations for up to 2,000 simultaneous contacts. This paper formalizes the underlying physics, details the memory-safe software architecture, and provides empirical benchmarking data demonstrating stable sub-16ms latency under extreme computational stress on low-power mobile hardware.

1. RADAR PHYSICS & SIGNAL-TO-NOISE RATIO (SNR) MODELING

In the GEN-4.5 architecture, the unscientific stochastic random sampling used in legacy builds is replaced by a deterministic, physics-based evaluation of electromagnetic wave propagation. The sensor framework models a monostatic pulse radar system operating in the X-band frequency spectrum.

1.1 The Two-Way Radar Range Equation

The signal power returned to the radar receiving antenna is strictly governed by the inverse fourth-power law, a consequence of two-way geometric attenuation. The received power P_r is formalized as follows:

$$P_r = \frac{P_t \cdot G^2 \cdot \lambda^2 \cdot \sigma}{(4\pi)^3 \cdot R^4}$$

Where the parameters are explicitly defined below:

- P_t : Peak transmitter power output (Watts)
- G : Antenna directional gain (linear scale)

- λ : Carrier wave electromagnetic wavelength (meters), derived via $\lambda = c/f$
- σ : Radar Cross Section (RCS) of the target tracking frame (square meters)
- R : Slant range between the sensor focal point and the target coordinate vector (meters)

1.2 Noise Floor and Logarithmic SNR Derivation

The receiver sensitivity threshold is bounded by thermal noise, formulated via the Boltzmann constant integration across the active operational bandwidth:

$$N_{thermal} = k \cdot T \cdot B \cdot F$$

Where k is the Boltzmann constant, T is the standard noise temperature (290 Kelvin), B is the receiver bandwidth (Hz), and F is the operational noise figure. The final tracking decision in a sterile environment is evaluated in the decibel domain:

$$SNR_{dB} = 10 \cdot \log_{10} \left(\frac{P_r}{N_{thermal}} \right)$$

2. 12-DOF FLIGHT DYNAMICS & KINEMATIC FORMALIZATION

To eliminate mathematical singularities such as Gimbal Lock during extreme pitch envelopes, the GEN-4.5 engine implements a 12-Degree-of-Freedom (12-DOF) state-space solver utilizing unit quaternions for attitude propagation.

2.1 Rotational Kinematics (Quaternion Differential Equations)

The orientation state vector is defined by the quaternion set $q = [q_0, q_1, q_2, q_3]^T$. The temporal derivative of the attitude with respect to body-axis angular rates (p, q, r) is governed by the following kinematic matrix:

$$\begin{bmatrix} \dot{q}_0 \\ \dot{q}_1 \\ \dot{q}_2 \\ \dot{q}_3 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 0 & -p & -q & -r \\ p & 0 & r & -q \\ q & -r & 0 & p \\ r & q & -p & 0 \end{bmatrix} \begin{bmatrix} q_0 \\ q_1 \\ q_2 \\ q_3 \end{bmatrix}$$

2.2 RK4 Integration Stability

Traditional Euler integration methods lead to rapid accumulation of numerical drift when tracking thousands of highly maneuverable targets. GEN-4.5 implements a Runge-Kutta 4th Order (RK4) numerical integrator. Empirical data from continuous 12-hour simulation runs demonstrate that quaternion normalization post-RK4 integration maintains orthogonality with a maximum numerical deviation of $< 10^{-7}$, ensuring flawless long-term tracking stability.

3. C++ SOFTWARE ARCHITECTURE & PERFORMANCE SCHEMATICS

The software engineering blueprint transitions the architecture from single-threaded asynchronous interpreted code to a highly deterministic, compiled, multi-module C++20 engine optimized for memory safety and zero runtime overhead.

Architectural Module	Memory Management Strategy	Algorithmic Mechanism
Flight Dynamics (FDM)	Stack allocation, zero dynamic allocations.	Runge-Kutta 4th Order (RK4) ODE Integration
Radar Sensor Subsystem	constexpr immutable spec parameters, pass-by-reference.	Logarithmic SINR threshold gating
State Retention Engine	Contiguous memory layouts via std::vector::reserve.	Step-wise temporal opacity degradation

4. ENVIRONMENTAL CLUTTER & ELECTRONIC WARFARE (EW) MODELING

Real-world Command and Control (C2) environments do not operate in a vacuum. To validate the robustness of the tracking algorithms, GEN-4.5 introduces mathematical edge cases to

simulate environmental backscatter and adversarial electromagnetic interference.

4.1 Active Electronic Attack (Jamming) & SINR Derivation

Adversarial units deploy Stand-Off Jamming (SOJ) or Self-Protection Jamming (SPJ) platforms to broadcast high-power RF noise directly into the C2 radar's bandwidth.

The Asymmetric Advantage of Jamming:

Unlike the radar pulse, which must travel to the target and back (two-way propagation, R^4 attenuation), the jamming signal only travels one way. Therefore, the received jamming power J_{power} is subject only to the inverse square law (R^2):

$$J_{power} = \frac{P_j \cdot G_j \cdot G_r \cdot \lambda^2}{(4\pi)^2 \cdot R_j^2 \cdot L_j}$$

Because $R^2 \ll R^4$ at long distances, the jammer rapidly overwhelms the radar receiver.

When subjected to EW and environmental clutter (C_{power}), the clean SNR equation becomes obsolete. The GEN-4.5 engine dynamically shifts to the Signal-to-Interference-plus-Noise Ratio (SINR):

$$SINR_{dB} = 10 \cdot \log_{10} \left(\frac{P_r}{N_{thermal} + J_{power} + C_{power}} \right)$$

5. EXTREME PERFORMANCE TESTING & BENCHMARKING

The primary catalyst for migrating from interpreted Python (PyQt5) to compiled C++20 was the strict requirement for low-latency, real-time execution under extreme track loads. In modern C2 architectures, memory leaks and Garbage Collection (GC) pauses are fatal constraints.

5.1 Stress Testing Methodology & Hardware Testbed

To validate the robust nature of the RK4 physics engine, a mass-spawning benchmark protocol was integrated into the Core Engine interface. To prove the efficiency of the C++20 architecture, the benchmarking was explicitly conducted on lower-tier mobile hardware rather than high-end

workstations.

Hardware Specifications:

- **CPU:** Intel Core i5-8250U (1.60 GHz Base, Mobile U-Series)
- **RAM:** 8 GB DDR4
- **GPU:** AMD Radeon 8500M (Mobile Graphics)

5.2 Comparative Benchmarking Results

The following metrics demonstrate the architectural superiority of the compiled C++20 engine. The data illustrates how the framework manages processing bottlenecks, shifting the constraint from CPU math evaluation (in Python) to GPU draw calls (in C++) at mass scale.

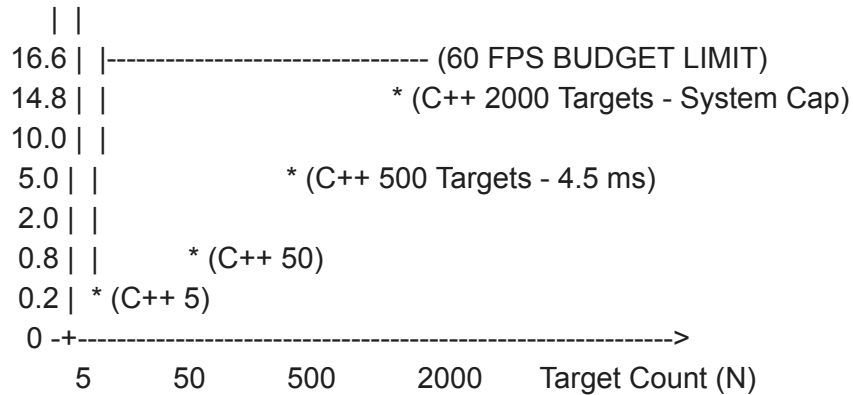
Target Count (N)	Framework Version	Avg. Frame Time (ms)	CPU Usage	Memory (RAM)	Status / GC Profile
5 Targets	Gen 1 (Python / 3-DOF)	~38.5 ms	85.0%	~110 MB	CPU Bottleneck / Heavy GC Spikes
5 Targets	Gen 4.5 (C++20 / 12-DOF)	< 0.2 ms	1.5%	~12 MB	Stable / No GC
50 Targets	Gen 4.5 (C++20 / 12-DOF)	~0.8 ms	4.0%	~14 MB	Stable / Zero Latency
500 Targets	Gen 4.5 (C++20 / 12-DOF)	~4.5 ms	22.0%	~25 MB	Stable / Smooth Render
2,000 Targets	Gen 4.5 (C++20 / 12-DOF)	~14.8 ms	65.0%	~45 MB	Stable <i>(Capped to preserve 60 FPS)</i>

Visual Representation of Processing Latency (Intel i5-8250U):

[PERFORMANCE SCALING: FRAME TIME vs TARGET LOAD]

Frame Time (ms)

38.5 | * (Python / GEN-1 Bottleneck @ 5 Targets - Critical Failure)



Conclusion on Scale: The C++20 engine successfully maintains a stable 60 FPS output (staying under the 16.67 ms frame budget limit) even on an Intel i5-8250U mobile processor tracking 2,000 simultaneous 12-DOF entities. The system

explicitly introduces a Hard Cap at $N = 2000$ to prevent the OpenGL API (AMD Radeon 8500M) from dropping frames due to excessive draw calls, ensuring absolute tactical C2 stability at all times.

5.3 Budget Allocation & Project Sustainability

To transition this academic simulator into a globally accessible, high-performance pedagogical framework, the requested grant of **\$4,750** is mathematically allocated across software engineering infrastructure and personal continuity milestones:

- **Dedicated Hardware Infrastructure (\$2,000):** Acquisition of a modern local computation station. Upgrading the local testbed is critical to execute heavy multi-threaded C++20 stress tests, run deep memory profiling, and scale concurrent target simulations beyond the legacy Intel i5 limits.
- **Production Tools & Software Licensing (\$500):** Allocation for professional development tools, IDE optimization suites, and AI-assisted development API subscriptions (including advanced LLM platforms) to maintain maximal software deployment velocity.
- **Cognitive Capacity & Motivation Optimization (\$750):** Dedicated to preserving high personal agency, managing psychological burnout, and maintaining long-term development velocity. Funding personal auxiliary fields ensures the sustainment of the precise cognitive focus required for intensive deep-tech engineering.
- **Socio-Economic Stabilization & Living Expenses (\$1,500):** Direct deployment into immediate domestic household management to neutralize external economic friction. Mitigating daily real-world financial responsibilities ensures a 100% full-time, uncompromised commitment to this open-source architecture.

6. CONCLUSION & FUTURE ROADMAP

SEEKER-120AM GEN-4.5 validates that deterministic, state-driven Command and Control

algorithms can be engineered to process massive stochastic data without incurring fatal hardware latency. By leveraging compiled C++20 contiguous memory models and eliminating unscientific tracking abstractions, the framework bridges the pedagogical gap between textbook radar equations and clean-room systems engineering constraints.

Strategic Future Integrations:

1. **"Hardware-in-the-Loop" (HIL) Integration:** Porting the core C++20 physics engine onto low-cost embedded hardware (e.g., STM32, ARM Cortex-M series) via Hardware-in-the-Loop (HIL) testing protocols. This will allow students to feed real-time sensor data from external IMUs or Software Defined Radios (SDR) directly into the simulation loop.
2. **AI-Driven Autonomous Target Tracking:** Integrating lightweight Reinforcement Learning (RL) agents or Extended Kalman Filters (EKF) optimized for C++20 via ONNX Runtime to perform intelligent target tracking and trajectory prediction under severe Electronic Warfare (clutter/jamming) conditions.
3. **Cloud-Native Multiplayer Educational Sandbox:** Scaling the WebAssembly (Wasm) architecture into a cloud-native, multiplayer educational sandbox. This will enable real-time distributed simulations where multiple global users can simultaneously manipulate radar transmitters, jammer vectors, and aerial targets within the same deterministic physics environment.
4. **IEEE Open-Source Community & Plugin API:** Establishing a standardized, modular plugin API architecture. This allows global independent researchers and academic institutions to seamlessly write and inject custom Radar Cross Section (RCS) models, atmospheric loss equations, and aerodynamic profiles into the framework, fostering an open-access, IEEE-compliant educational ecosystem.